

Recursion

Rizwan Rashid



Edit with WPS Office

Recursion

- Recursion in java is a process in which a method calls itself continuously
- A method in java that calls itself is called recursive method
- It makes the code compact but complex to understand
- Syntax:

```
returntype methodname(){  
    //code to be executed  
    methodname();//calling same method  
}
```



Example-1 (Infinite Times)

```
public static void main(String[] args){  
    infiniteMethod();  
}  
public static void infiniteMethod(){  
    System.out.println("infinite Recursion");  
    infiniteMethod();  
}
```



Example-2 (Finite Times)

```
static int count=0;
public static void main(String[] args){
    finiteMethod();
}
public static void finiteMethod(){
    if(count<=5){
        System.out.println(count+" Finite Method");
        count++;
        finiteMethod();
    }
}
```



Characteristics of Recursion

All recursive methods have the following characteristics:

- One or more base cases (the simplest case) are used to stop recursion.
- Every recursive call reduces the original problem, bringing it increasingly closer to a base case until it becomes that case.

In general, to solve a problem using recursion, you break it into subproblems. If a subproblem resembles the original problem, you can apply the same approach to solve the subproblem recursively. This subproblem is almost the same as the original problem in nature with a smaller size.



Computing Factorial

`factorial(0) = 1;`

`factorial(n) = n*factorial(n-1);`

$n! = n * (n-1)!$

$0! = 1$



Computing Factorial

factorial(4)

factorial(0) = 1;
factorial(n) = n*factorial(n-1);



Computing Factorial

$$\text{factorial}(4) = 4 * \text{factorial}(3)$$

$$\text{factorial}(0) = 1;$$

$$\text{factorial}(n) = n * \text{factorial}(n - 1);$$



Computing Factorial

$$\begin{aligned}\text{factorial}(4) &= 4 * \text{factorial}(3) \\ &= 4 * 3 * \text{factorial}(2)\end{aligned}$$

`factorial(0) = 1;`
`factorial(n) = n*factorial(n-1);`



Computing Factorial

$$\begin{aligned}\text{factorial}(4) &= 4 * \text{factorial}(3) \\ &= 4 * 3 * \text{factorial}(2) \\ &= 4 * 3 * (2 * \text{factorial}(1))\end{aligned}$$

`factorial(0) = 1;`
`factorial(n) = n*factorial(n-1);`



Computing Factorial

`factorial(0) = 1;`

`factorial(n) = n*factorial(n-1);`

`factorial(4) = 4 * factorial(3)`

`= 4 * 3 * factorial(2)`

`= 4 * 3 * (2 * factorial(1))`

`= 4 * 3 * ( 2 * (1 * factorial(0)))`



Computing Factorial

`factorial(0) = 1;`

`factorial(n) = n*factorial(n-1);`

`factorial(4) = 4 * factorial(3)`

`= 4 * 3 * factorial(2)`

`= 4 * 3 * (2 * factorial(1))`

`= 4 * 3 * ( 2 * (1 * factorial(0)))`

`= 4 * 3 * ( 2 * ( 1 * 1)))`



Computing Factorial

`factorial(0) = 1;`

`factorial(n) = n*factorial(n-1);`

`factorial(4) = 4 * factorial(3)`

`= 4 * 3 * factorial(2)`

`= 4 * 3 * (2 * factorial(1))`

`= 4 * 3 * ( 2 * (1 * factorial(0)))`

`= 4 * 3 * ( 2 * ( 1 * 1)))`

`= 4 * 3 * ( 2 * 1)`



Computing Factorial

`factorial(0) = 1;`

`factorial(n) = n*factorial(n-1);`

$$\text{factorial}(4) = 4 * \text{factorial}(3)$$

$$= 4 * 3 * \text{factorial}(2)$$

$$= 4 * 3 * (2 * \text{factorial}(1))$$

$$= 4 * 3 * (2 * (1 * \text{factorial}(0)))$$

$$= 4 * 3 * (2 * (1 * 1))$$

$$= 4 * 3 * (2 * 1)$$

$$= 4 * 3 * 2$$



Computing Factorial

`factorial(0) = 1;`

`factorial(n) = n*factorial(n-1);`

$$\begin{aligned}\text{factorial}(4) &= 4 * \text{factorial}(3) \\ &= 4 * (3 * \text{factorial}(2)) \\ &= 4 * (3 * (2 * \text{factorial}(1))) \\ &= 4 * (3 * (2 * (1 * \text{factorial}(0)))) \\ &= 4 * (3 * (2 * (1 * 1))) \\ &= 4 * (3 * (2 * 1)) \\ &= 4 * (3 * 2) \\ &= 4 * (6)\end{aligned}$$



Computing Factorial

factorial(0) = 1;

factorial(n) = n*factorial(n-1);

$$\begin{aligned}\text{factorial}(4) &= 4 * \text{factorial}(3) \\ &= 4 * (3 * \text{factorial}(2)) \\ &= 4 * (3 * (2 * \text{factorial}(1))) \\ &= 4 * (3 * (2 * (1 * \text{factorial}(0)))) \\ &= 4 * (3 * (2 * (1 * 1))) \\ &= 4 * (3 * (2 * 1)) \\ &= 4 * (3 * 2) \\ &= 4 * (6) \\ &= 24\end{aligned}$$



Trace Recursive factorial

factorial(4)

Executes factorial(4)

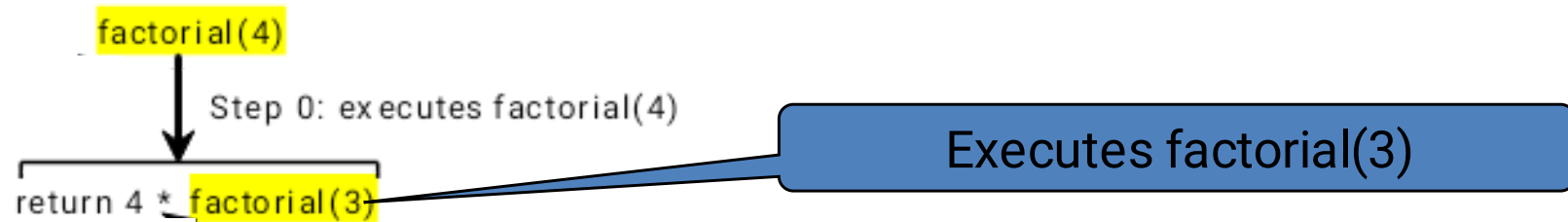
Stack

Space Required
for factorial(4)

Main method



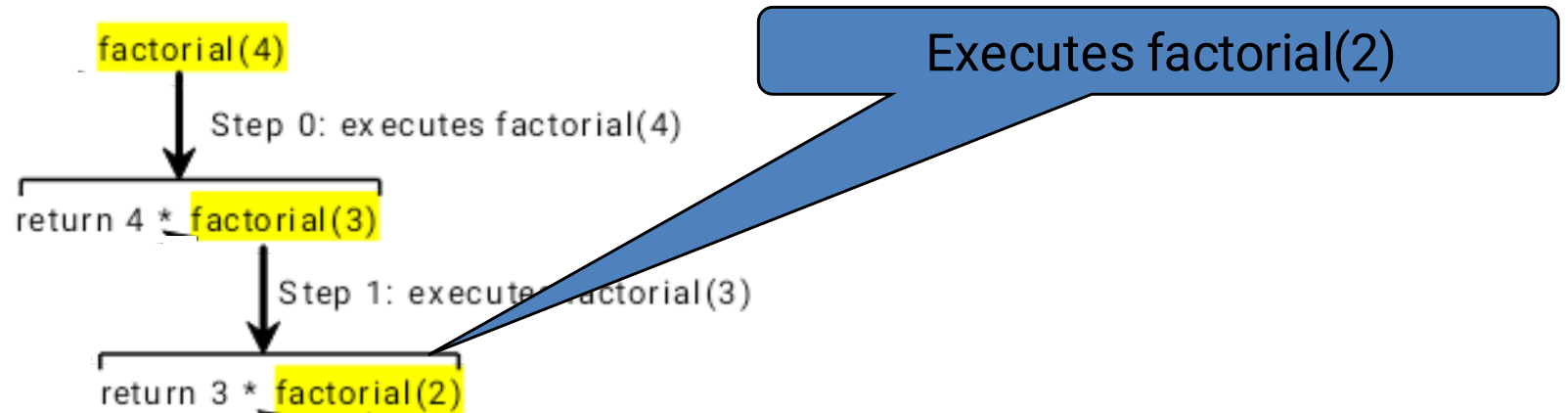
Trace Recursive factorial



Stack
Space Required for factorial(3)
Space Required for factorial(4)
Main method



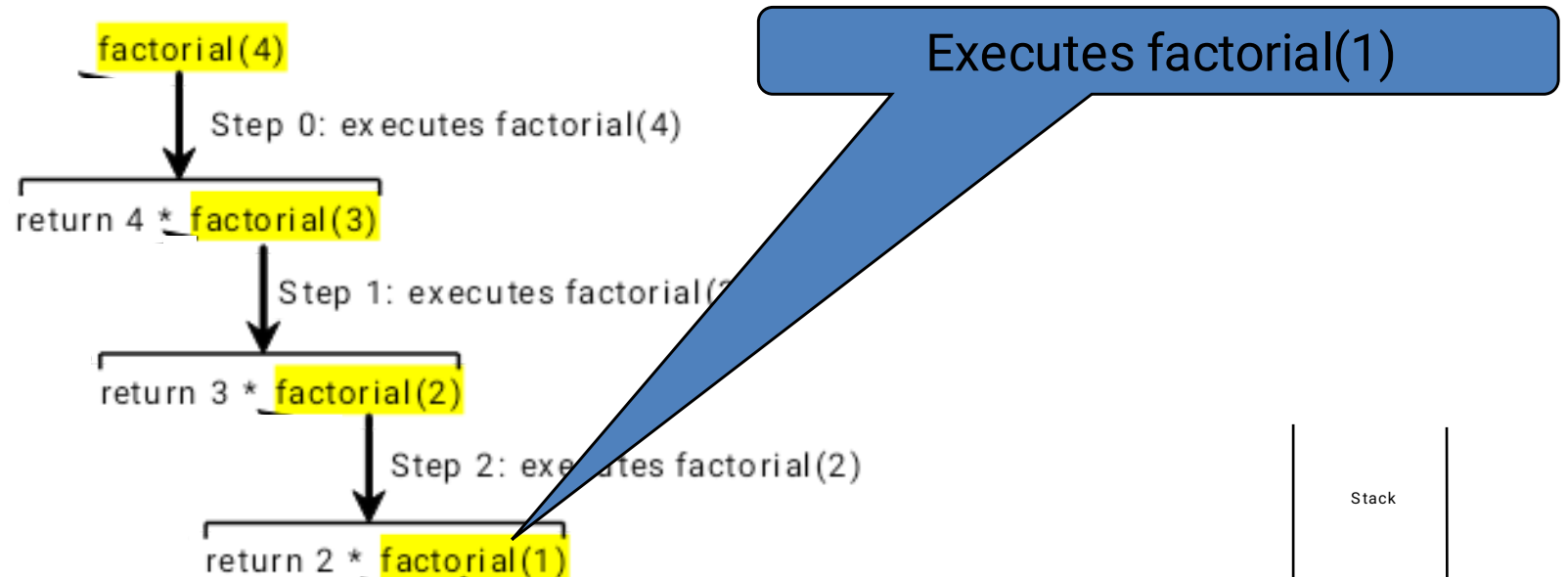
Trace Recursive factorial



Stack
Space Required for factorial(2)
Space Required for factorial(3)
Space Required for factorial(4)
Main method



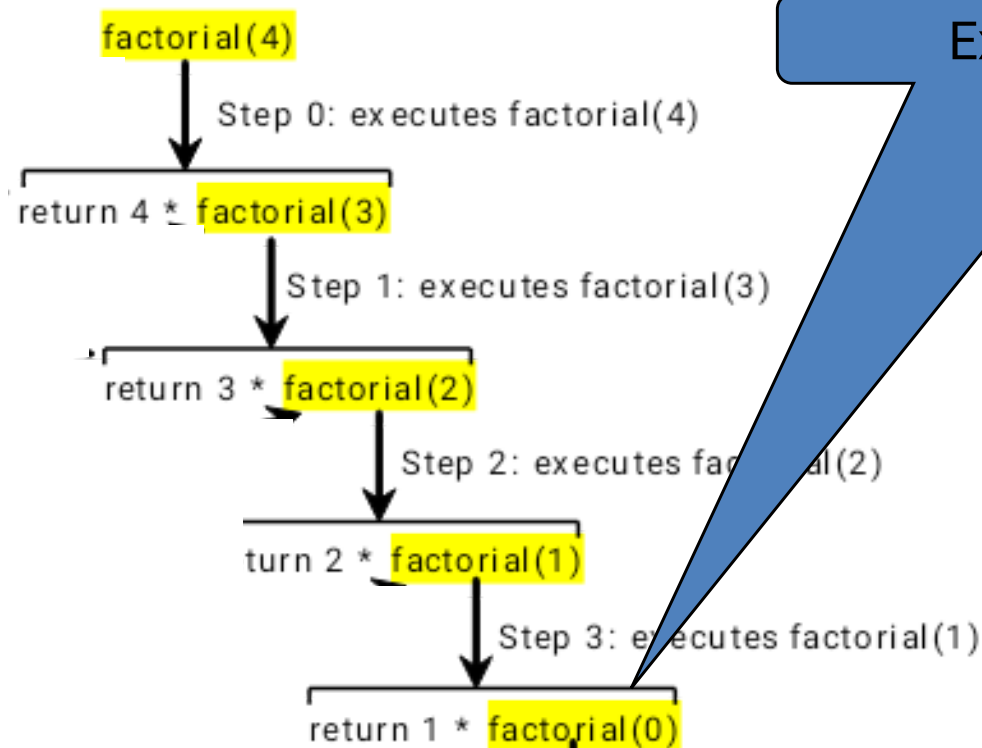
Trace Recursive factorial



Stack
Space Required for factorial(1)
Space Required for factorial(2)
Space Required for factorial(3)
Space Required for factorial(4)
Main method



Trace Recursive factorial

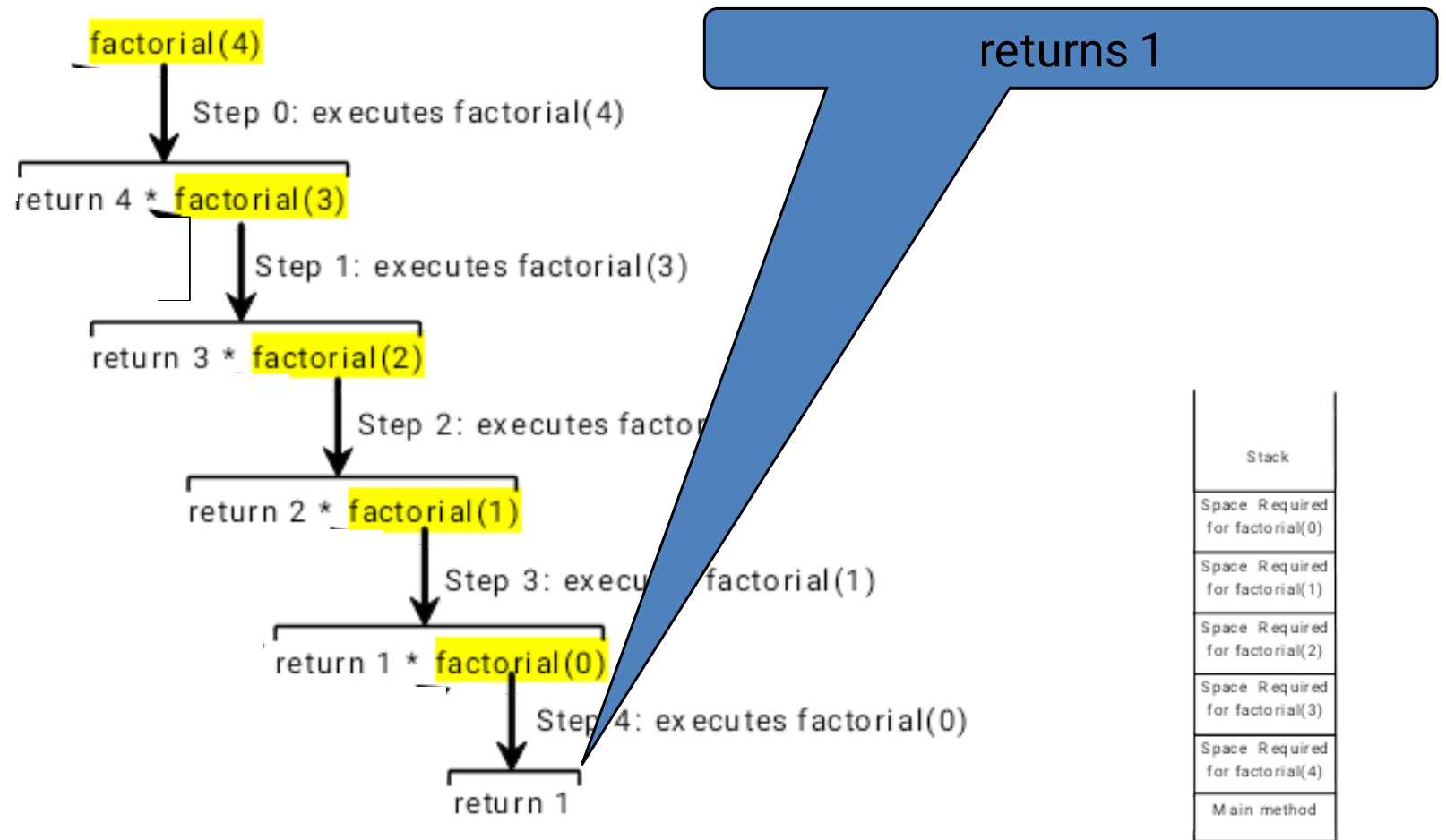


Executes `factorial(0)`

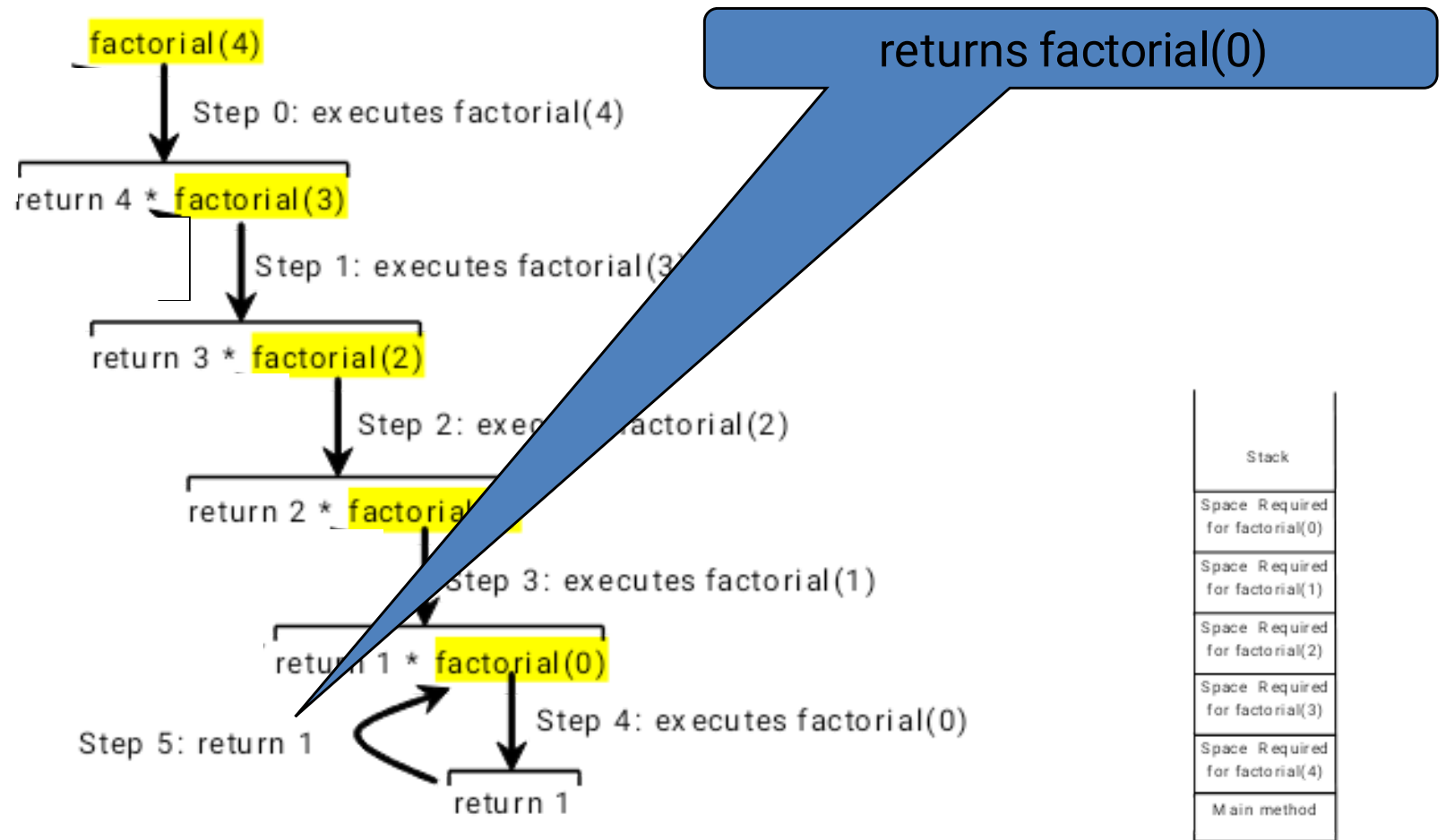
Stack
Space Required for factorial(0)
Space Required for factorial(1)
Space Required for factorial(2)
Space Required for factorial(3)
Space Required for factorial(4)
Main method



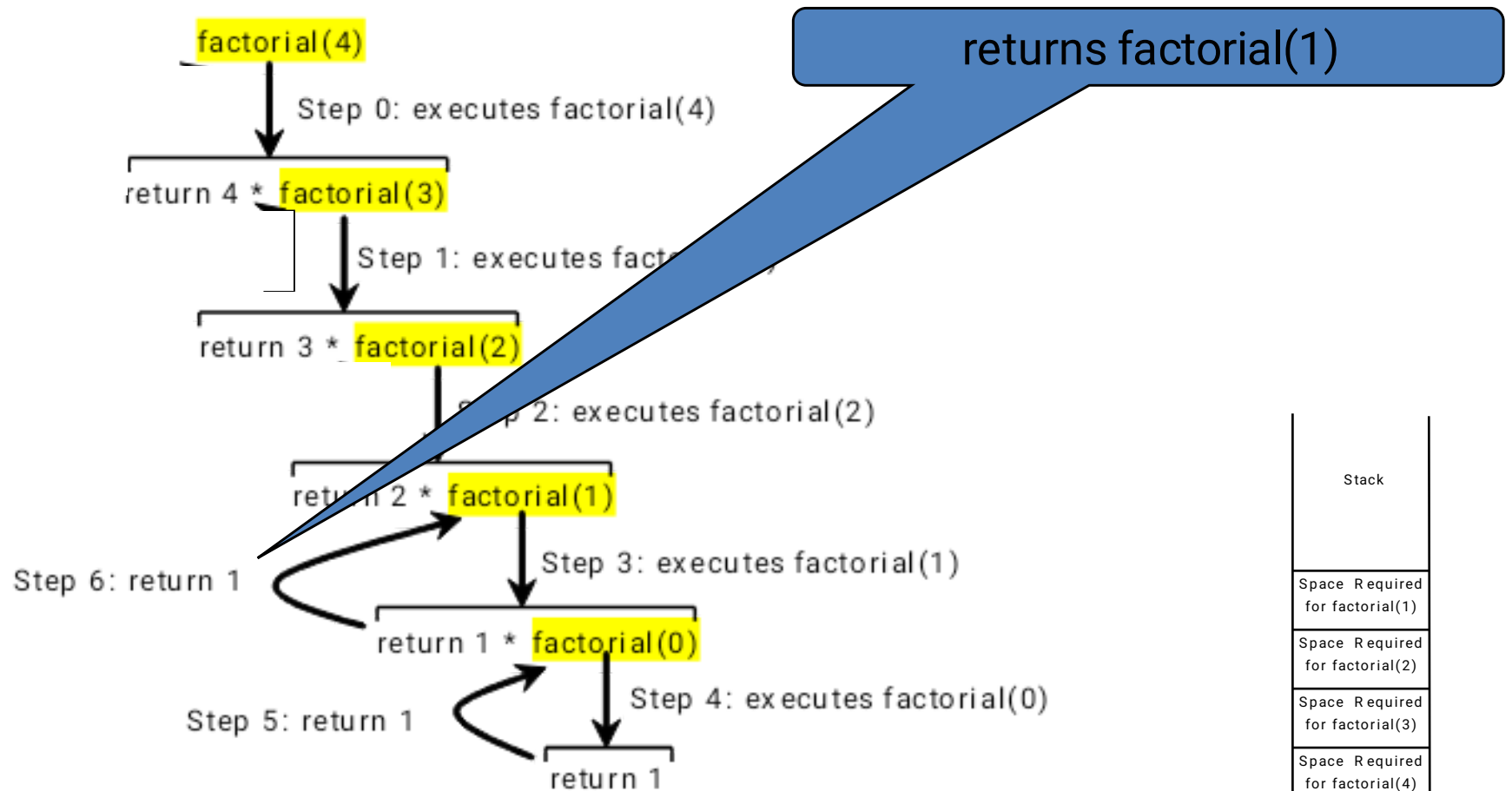
Trace Recursive factorial



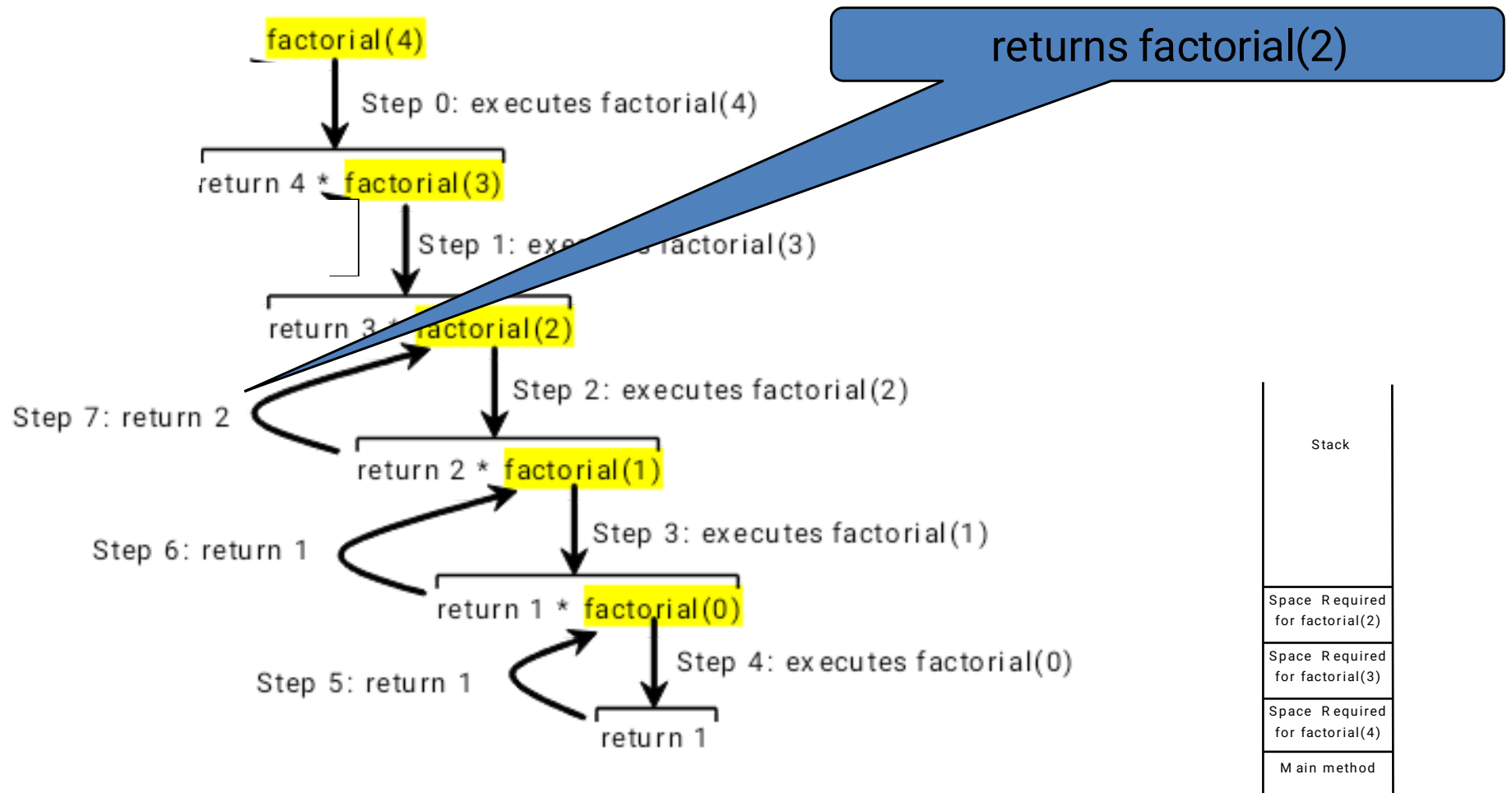
Trace Recursive factorial



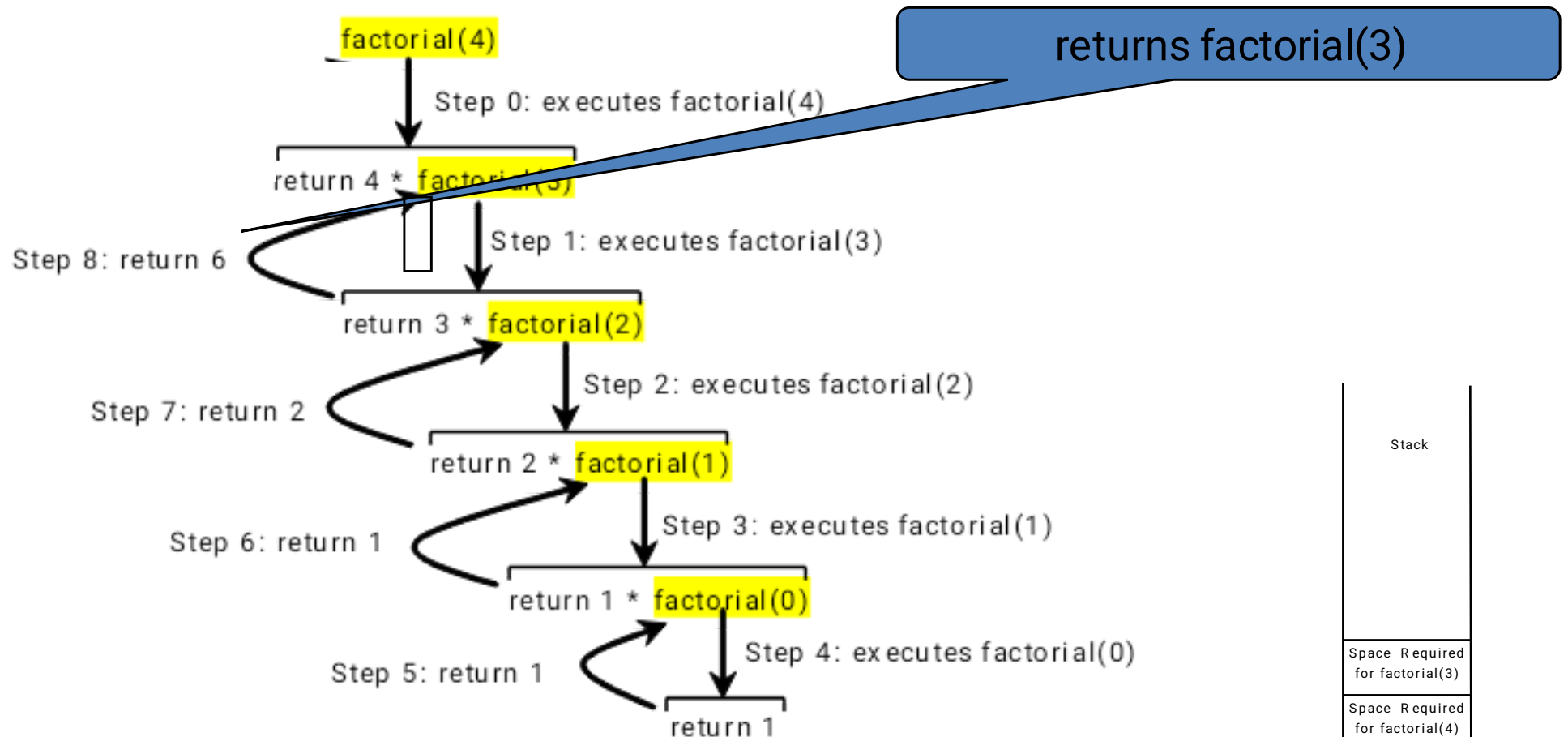
Trace Recursive factorial



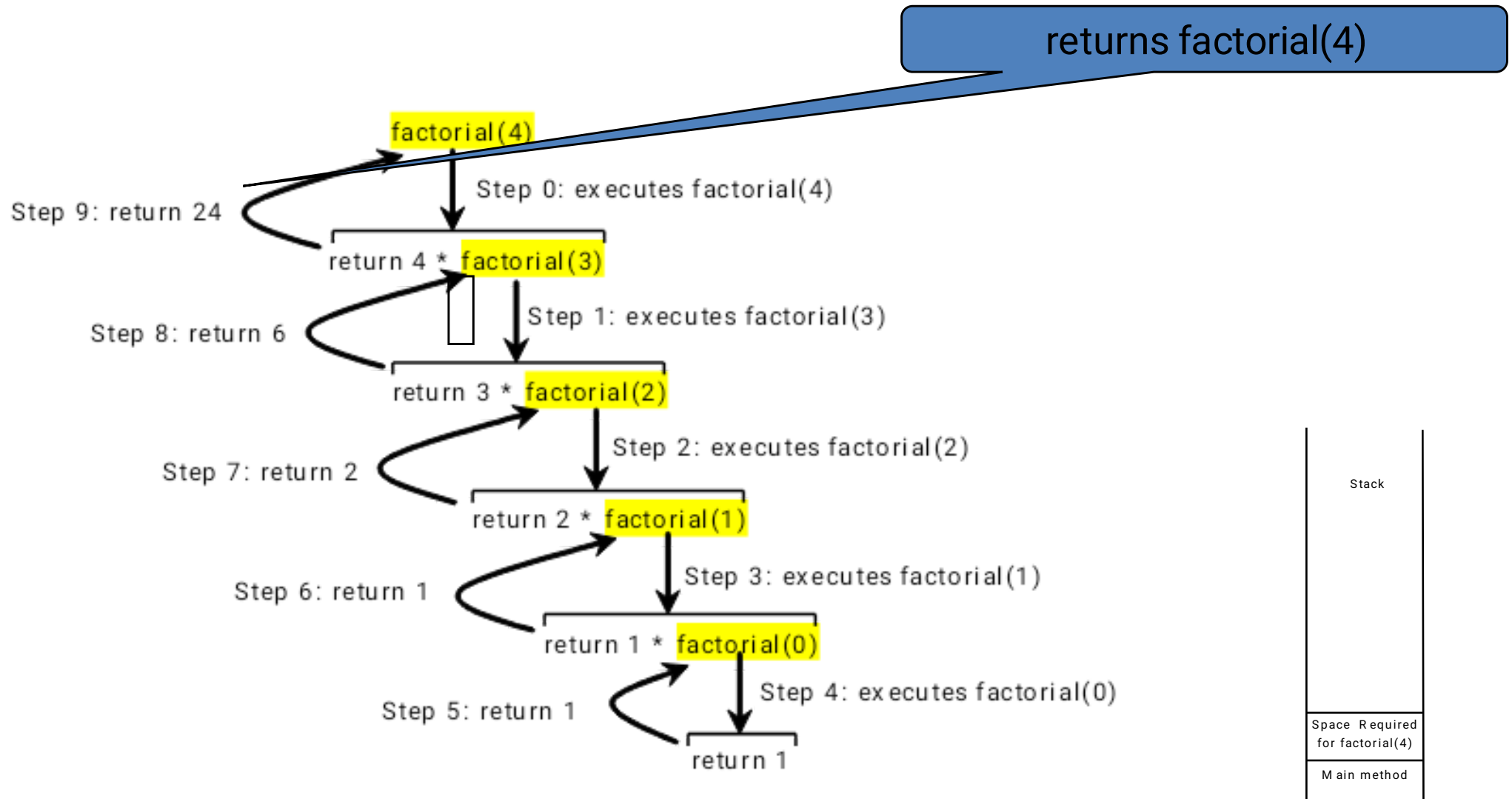
Trace Recursive factorial



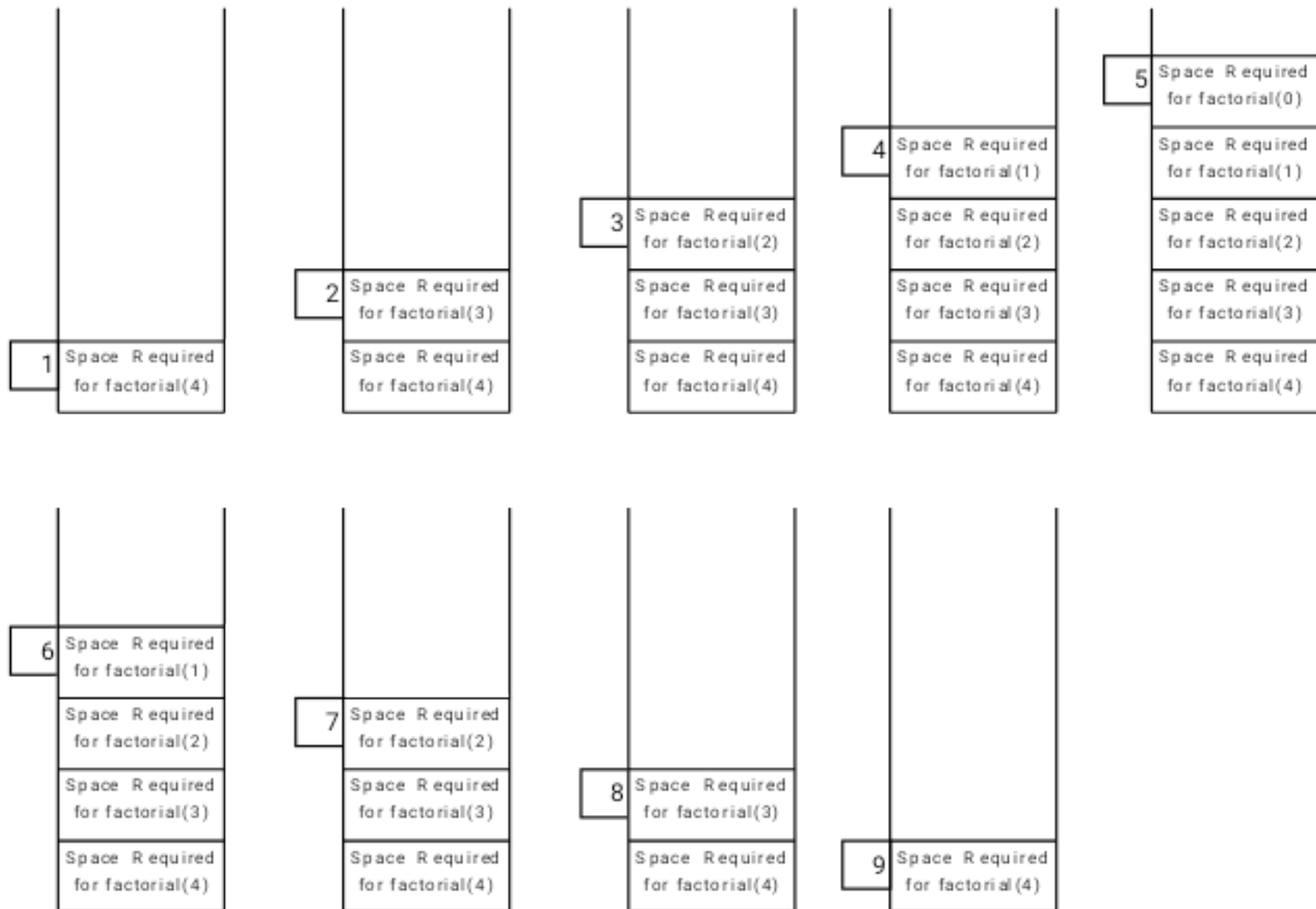
Trace Recursive factorial



Trace Recursive factorial



factorial(4) Stack Trace



Fibonacci Numbers

Fibonacci series: 0 1 1 2 3 5 8 13 21 34 55 89...

indices: 0 1 2 3 4 5 6 7 8 9 10 11

$$\text{fib}(0) = 0;$$

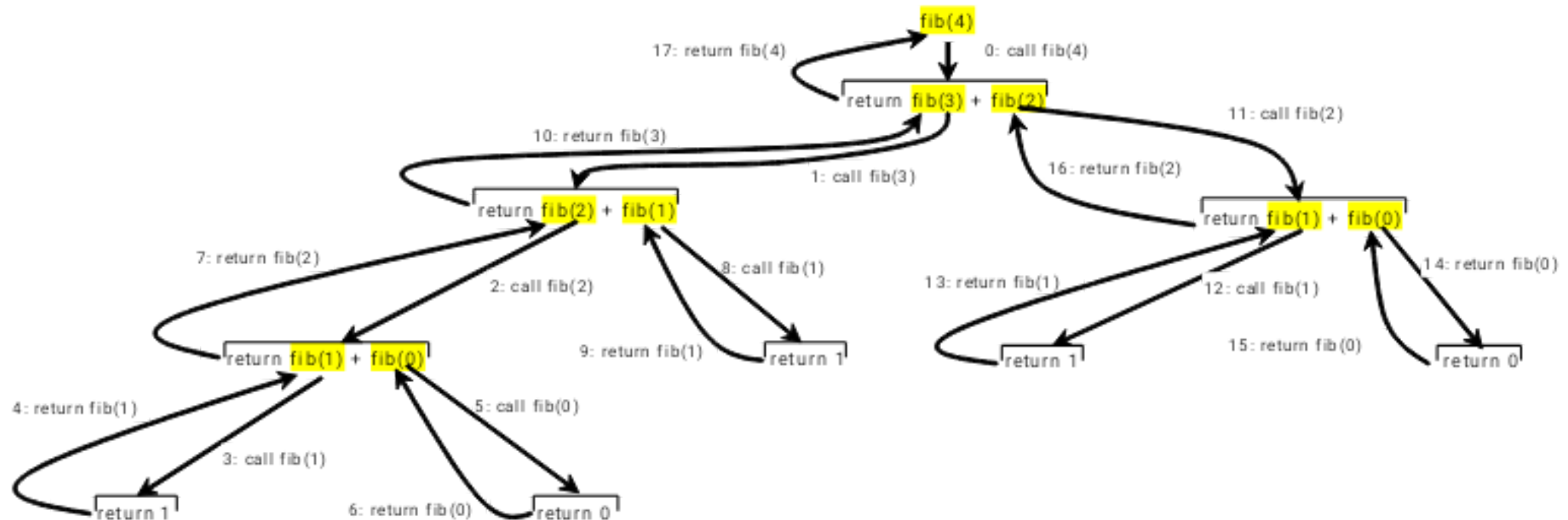
$$\text{fib}(1) = 1;$$

$$\text{fib}(\text{index}) = \text{fib}(\text{index} - 1) + \text{fib}(\text{index} - 2); \text{index} \geq 2$$

$$\begin{aligned} \text{fib}(3) &= \text{fib}(2) + \text{fib}(1) = (\text{fib}(1) + \text{fib}(0)) + \text{fib}(1) = (1 + 0) + \text{fib}(1) \\ &= 1 + \text{fib}(1) = 1 + 1 = 2 \end{aligned}$$



Fibonacci Numbers, cont.



LISTING 18.2 ComputeFibonacci.java

```
1  import java.util.Scanner;
2
3  public class ComputeFibonacci {
4      /** Main method */
5      public static void main(String[] args) {
6          // Create a Scanner
7          Scanner input = new Scanner(System.in);
8          System.out.print("Enter an index for a Fibonacci number: ");
9          int index = input.nextInt();
10
11         // Find and display the Fibonacci number
12         System.out.println("The Fibonacci number at index "
13             + index + " is " + fib(index));
14     }
15
16     /** The method for finding the Fibonacci number */
17     public static long fib(long index) {
18         if (index == 0) // Base case
19             return 0;
20         else if (index == 1) // Base case
21             return 1;
22         else // Reduction and recursive calls
23             return fib(index - 1) + fib(index - 2);
24     }
25 }
```



Think Recursively

Many of the problems presented in the early chapters can be solved using recursion if you *think recursively*. For example, the palindrome problem can be solved recursively as follows:

```
public static boolean isPalindrome(String s) {  
    if (s.length() <= 1) // Base case  
        return true;  
    else if (s.charAt(0) != s.charAt(s.length() - 1)) // Base case  
        return false;  
    else  
        return isPalindrome(s.substring(1, s.length() - 1));  
}
```



Recursive Helper Methods

The preceding recursive `isPalindrome` method is not efficient, because it creates a new string for every recursive call. To avoid creating new strings, use a **helper method**:

```
public static boolean isPalindrome(String s) {  
    return isPalindrome(s, 0, s.length() - 1);  
}  
  
public static boolean isPalindrome(String s, int low, int high) {  
    if (high <= low) // Base case  
        return true;  
    else if (s.charAt(low) != s.charAt(high)) // Base case  
        return false;  
    else  
        return isPalindrome(s, low + 1, high - 1);  
}
```



Recursive Selection Sort

1. Find the smallest number in the list and swaps it with the first number.
2. Ignore the first number and sort the remaining smaller list recursively.



Recursive Binary Search

1. Case 1: If the key is less than the middle element, recursively search the key in the first half of the array.
2. Case 2: If the key is equal to the middle element, the search ends with a match.
3. Case 3: If the key is greater than the middle element, recursively search the key in the second half of the array.



Recursive Implementation

```
/** Use binary search to find the key in the list */
public static int recursiveBinarySearch(int[] list, int key) {
    int low = 0;
    int high = list.length - 1;
    return recursiveBinarySearch(list, key, low, high);
}

/** Use binary search to find the key in the list between
    list[low] list[high] */
public static int recursiveBinarySearch(int[] list, int key,
    int low, int high) {
    if (low > high) // The list has been exhausted without a match
        return -low - 1;

    int mid = (low + high) / 2;
    if (key < list[mid])
        return recursiveBinarySearch(list, key, low, mid - 1);
    else if (key == list[mid])
        return mid;
    else
        return recursiveBinarySearch(list, key, mid + 1, high);
}
```



Recursion vs. Iteration

Recursion is an alternative form of program control. It is essentially repetition without a loop.

Recursion bears substantial overhead. Each time the program calls a method, the system must assign space for all of the method's local variables and parameters. This can consume considerable memory and requires extra time to manage the additional space.



Tail Recursion

A recursive method is said to be *tail recursive* if there are no pending operations to be performed on return from a recursive call.

Recursive method A

...

...

...

Invoke method A recursively

(a) Tail recursion

Recursive method B

...

...

Invoke method B recursively

...

...

(b) Nontail recursion



Tail recursive

LISTING 18.10 ComputeFactorialTailRecursion.java

```
1 public class ComputeFactorialTailRecursion {
2     /** Return the factorial for a specified number */
3     public static long factorial(int n) {
4         return factorial(n, 1); // Call auxiliary method
5     }
6
7     /** Auxiliary tail-recursive method for factorial */
8     private static long factorial(int n, int result) {
9         if (n == 0)
10             return result;
11         else
12             return factorial(n - 1, n * result); // Recursive call
13     }
14 }
```

Non-tail recursive

```
16 public static long factorial(int n) {
17     if (n == 0) // Base case
18         return 1;
19     else
20         return n * factorial(n - 1); // Recursive call
21 }
```

